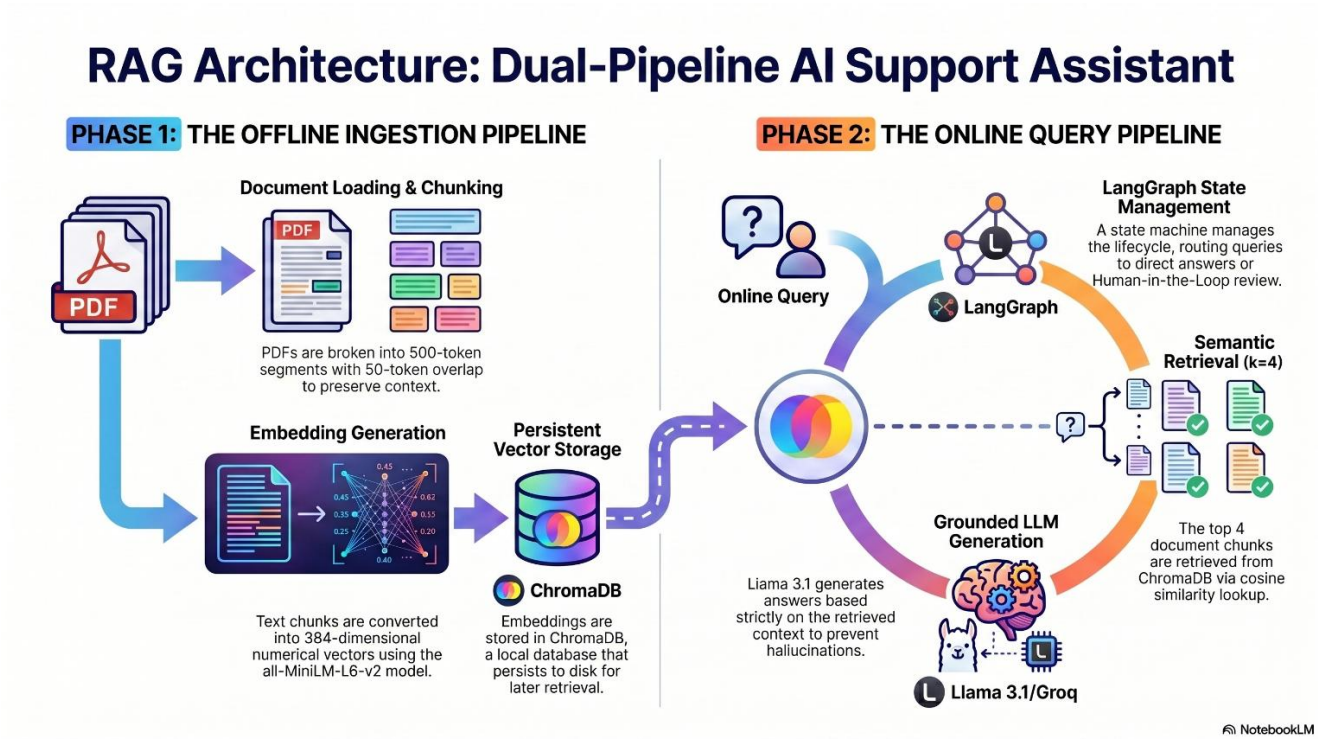
HIGH-LEVEL DESIGN

RAG-Based Customer Support Assistant with LangGraph & HITL

Document	High-Level Design — RAG Customer Support Assistant
Intern	Zaara Shaikh | ID: IN226050502
Org	Innomatics Research Labs — Advanced GenAI Internship Feb 2026
Date	April 23, 2026



1. System Overview

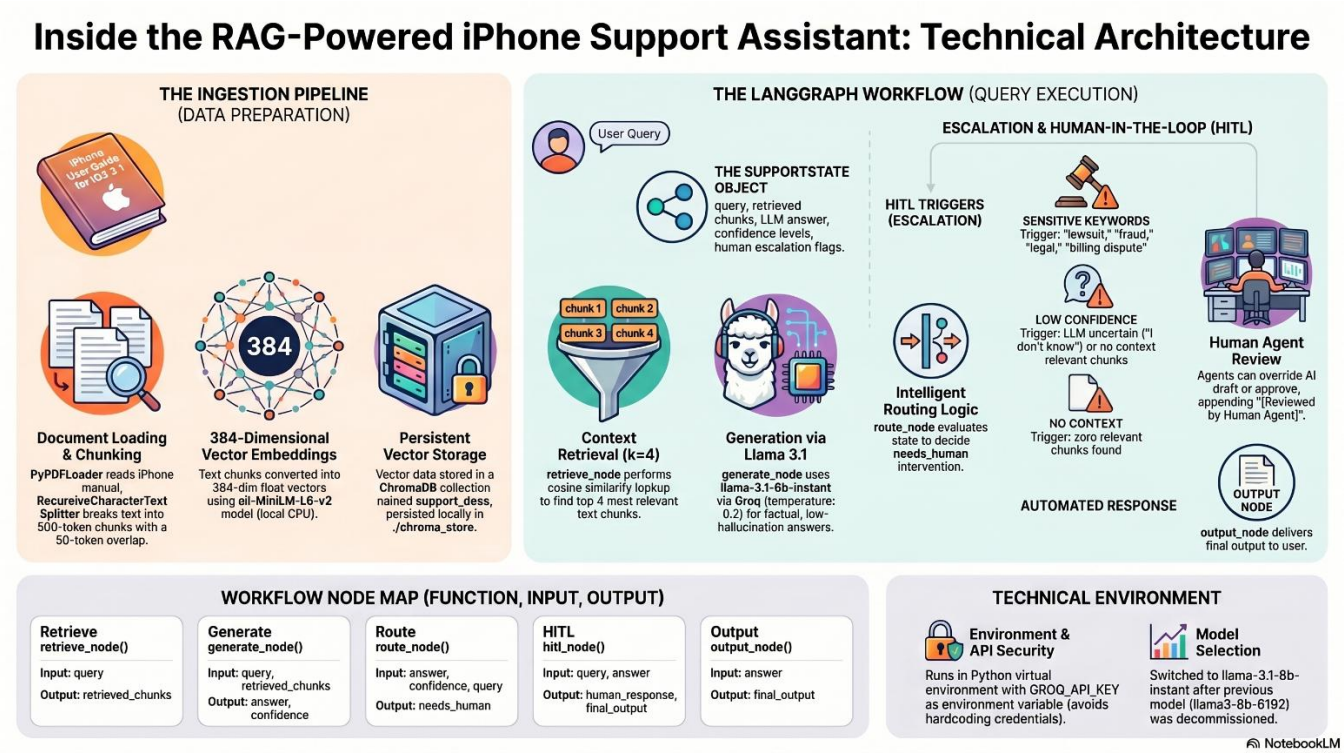
1.1 Problem Definition

Customer support teams handle a high volume of repetitive queries — return policies, troubleshooting steps, subscription FAQs. Traditional chatbots rely on rigid rule-based scripts that break on any variation. This system uses RAG: the bot answers from actual company documentation, not hardcoded responses. It only escalates to a human when it genuinely cannot handle something reliably — for example, legal queries or questions outside the knowledge base.

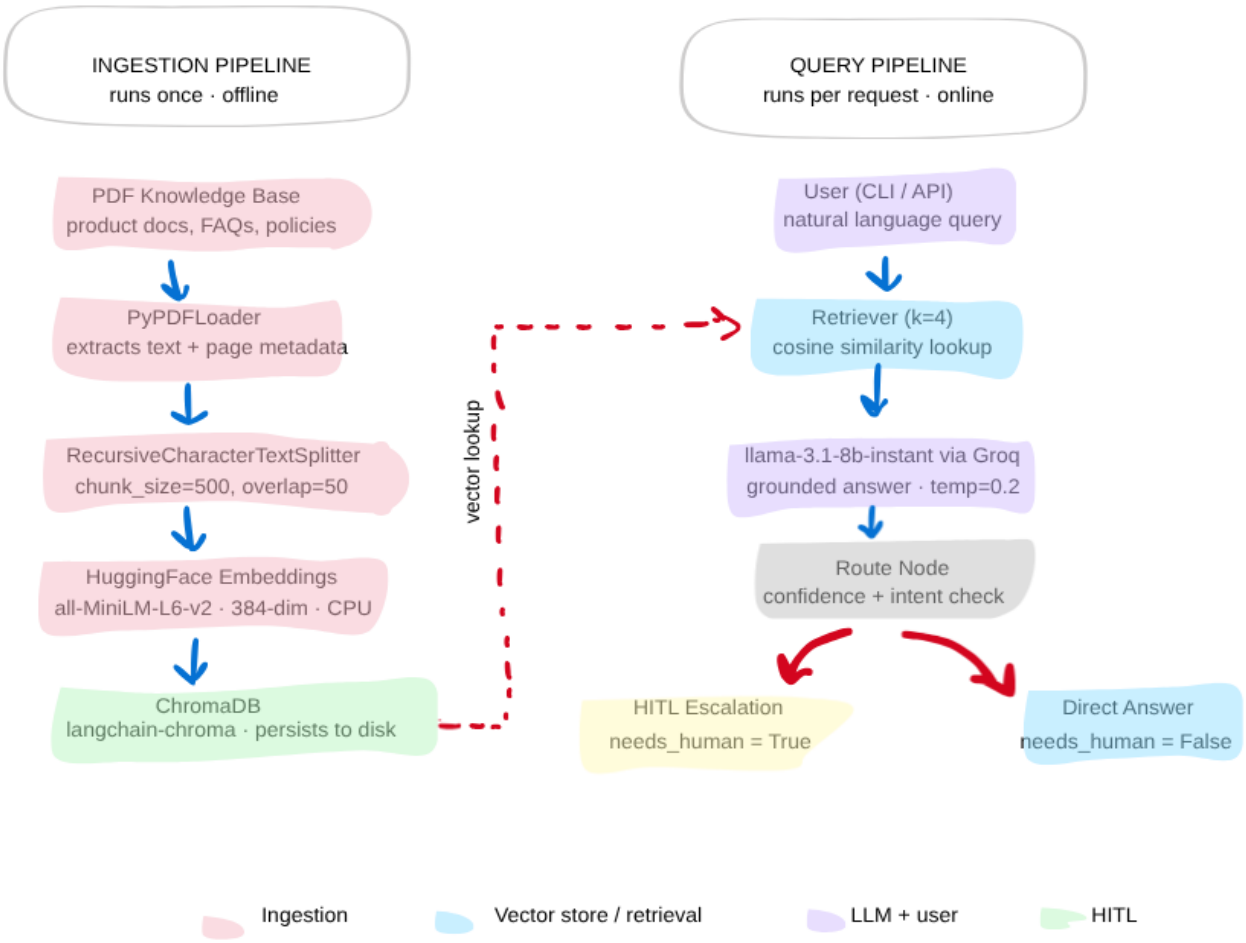
1.2 Scope

- Ingests one or more PDF documents as the knowledge source
- Handles customer queries via CLI interface (extensible to REST API)
- Retrieves semantically relevant chunks and generates grounded answers
- Uses LangGraph to manage the query lifecycle as a typed state machine
- Routes to Human-in-the-Loop (HITL) when confidence is low, query is sensitive (e.g. legal, fraud), or context is missing
- Tested against real queries: routine support (high confidence → direct) and legal queries (→ HITL escalation)
- Does NOT include real-time web search, multi-turn memory, or user auth (future scope).

2. Architecture Diagram



The system is split into two pipelines. Ingestion (left) runs once offline to build the knowledge base. The query pipeline (right) runs per user request. ChromaDB is shared between them via the vector lookup path (dashed arrow).



Data Flow — Ingestion

Step 1: PDF passed to PyPDFLoader → raw text extracted page-by-page with source/page metadata

Step 2: langchain_text_splitters.RecursiveCharacterTextSplitter → ~500-token chunks, 50 overlap

Step 3: all-MiniLM-L6-v2 (sentence-transformers) → 384-dim embeddings, CPU-only

Step 4: Embeddings stored in ChromaDB via langchain-chroma package, persisted to ./chroma_store

Data Flow — Query

Step 1: User submits natural language query

Step 2: Query embedded, top-4 chunks retrieved from ChromaDB by cosine similarity

Step 3: Chunks + query sent to llama-3.1-8b-instant via Groq (temperature=0.2)

Step 4: LangGraph route_node checks confidence level and query intent

Step 5: High confidence / routine → direct answer. Low confidence / legal / fraud → HITL

Step 6: Final answer returned with [AI Assistant] or [Reviewed by Human Agent] tag

3. Component Description

Document Loader

PyPDFLoader from langchain-community. Handles multi-page PDFs, returns LangChain Document objects with page_content and metadata dict (source filename, page number).

Chunking Strategy

langchain_text_splitters.RecursiveCharacterTextSplitter with chunk_size=500, overlap=50. Recursive separator list [\n\n, \n, '.', ' '] respects paragraph and sentence boundaries before falling through to character-level splits.

Embedding Model

all-MiniLM-L6-v2 via HuggingFaceEmbeddings (sentence-transformers). Produces 384-dim vectors, runs entirely on CPU — no GPU, no API key required. Selected for best quality-to-size ratio on semantic similarity benchmarks within free-tier constraints.

Vector Store (ChromaDB via langchain-chroma)

langchain-chroma package (updated from deprecated langchain_community.vectorstores.Chroma). Persists embeddings + metadata to disk. No external infrastructure needed for prototype scope.

Retriever

Similarity retriever fetching top k=4 chunks. 4 balances context quality vs prompt length — too many chunks dilutes relevance and increases LLM latency.

LLM — llama-3.1-8b-instant via Groq

Llama 3.1 was selected after the original llama3-8b-8192 model was decommissioned by Groq's API during development (HTTP 400). Llama 3.1 provides better instruction following and stable API availability. Groq's inference speed (~500 tok/s) keeps response time under 2 seconds.

Graph Workflow Engine (LangGraph)

StateGraph with 5 typed nodes. Each node reads from and writes to a shared SupportState TypedDict. Conditional edges at route_node dispatch dynamically without if-else chains in the main loop.

Routing Layer

route_node checks: (a) LLM confidence from output text keywords, (b) presence of sensitive keywords in query (lawsuit, fraud, legal, billing dispute), (c) empty retrieved_chunks list. Sets needs_human boolean. Testing confirmed correct escalation on legal queries.

HITL Module

hitl_node displays AI draft to agent, accepts override input. CLI-based for this implementation. Production equivalent: LangGraph interrupt + ticketing system API (Zendesk/Freshdesk) + state persistence in PostgreSQL for async pause/resume.

4. Data Flow

4.1 How Data Moves from PDF → Answer

The entire system runs in two phases. First, a one-time offline ingestion that builds the knowledge base. Second, a real-time query pipeline that runs on every user request.

Ingestion Phase (offline, runs once):

Step 1 — A PDF file is passed to PyPDFLoader. It reads the file page by page and produces a list of LangChain Document objects. Each Document carries two things: page_content (the raw extracted text) and metadata (source filename and page number).

Step 2 — These Documents are passed to RecursiveCharacterTextSplitter. It breaks each page's content into chunks of approximately 500 tokens with a 50-token overlap. The splitter tries to cut at paragraph breaks first (\n\n), then line breaks (\n), then sentence endings (.), and only splits mid-word as a last resort. This preserves semantic meaning.

Step 3 — Each chunk is passed through HuggingFaceEmbeddings using the all-MiniLM-L6-v2 model. This model converts each text chunk into a 384-dimensional float vector — a numeric representation of its meaning in semantic space.

Step 4 — These vectors, along with their original text and metadata, are stored in ChromaDB via the langchain-chroma package. ChromaDB automatically persists them to the ./chroma_store directory on disk. This step only runs once unless the knowledge base needs to be updated.

Query Phase (online, per request):

Step 1 — The user types a natural language query at the CLI.

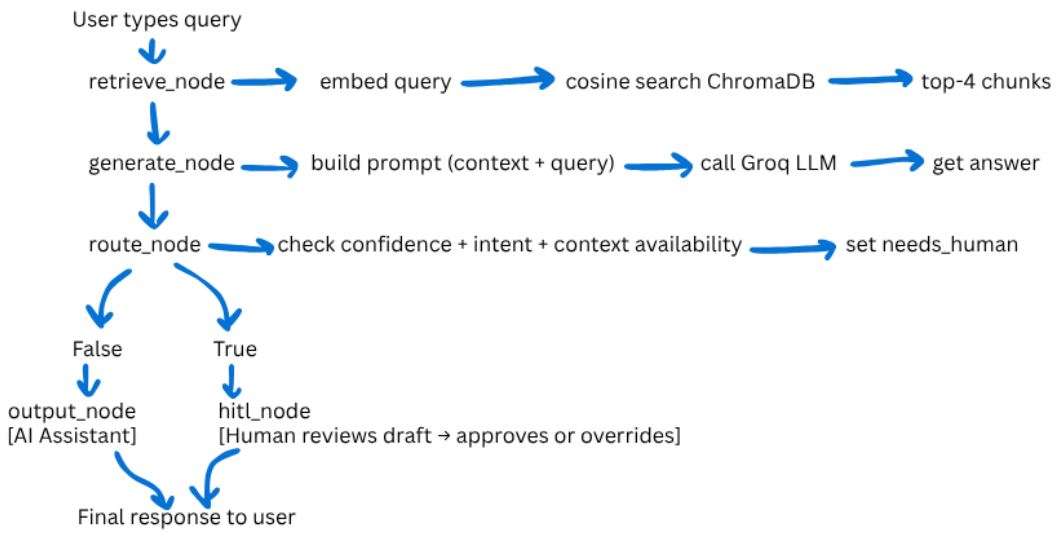
Step 2 — The query string is passed to the retrieve_node in the LangGraph graph. The retriever embeds the query using the same all-MiniLM-L6-v2 model (so it exists in the same vector space as the stored chunks) and performs a cosine similarity search in ChromaDB. It fetches the top 4 most semantically similar chunks.

Step 3 — The 4 retrieved chunks are concatenated as context and combined with the original query into a structured prompt. This prompt is sent to llama-3.1-8b-instant via the Groq API. The model is instructed to answer only from the provided context and to admit when it doesn't know.

Step 4 — The generated answer passes to route_node, which checks: (a) whether the answer contains low-confidence phrases, (b) whether the query contains sensitive keywords like "lawsuit" or "fraud", and (c) whether any chunks were retrieved at all. It sets a needs_human boolean flag.

Step 5 — If needs_human is False, the answer goes to output_node, which prepends [AI Assistant] and returns it to the user. If True, it goes to hitl_node, where a human agent reviews the AI draft and either approves or overrides it.

4.2 Query Lifecycle



5. Technology Choices

Technology	Choice	Reasoning
Vector DB	ChromaDB (langchain-chroma)	Free, local persistence, no external infrastructure needed. Migrated from the deprecated langchain_community.vectorstores.Chroma import to the updated langchain-chroma package during development.
Workflow	LangGraph	Built specifically for LLM state machines. Provides clean node/edge model, native conditional routing, typed state dict, and integrates directly with LangChain ecosystem.
LLM	llama-3.1-8b-instant via Groq	Switched from llama3-8b-8192 which was decommissioned by Groq mid-development (HTTP 400). Llama 3.1 offers improved instruction following and API stability. Groq provides fastest free-tier inference at ~500 tokens/second.

Embeddings	all-MiniLM-L6-v2 (sentence-transformers)	Best quality-to-size ratio for semantic similarity. 80MB model, runs entirely on CPU, no API key required, no cost per call.
PDF Loader	LangChain PyPDFLoader (langchain-community)	Returns native LangChain Document objects with page metadata preserved. Handles multi-page PDFs cleanly.
Text Splitter	langchain-text-splitters	Updated modular package post LangChain 0.2.9 restructure. Replaces deprecated langchain.text_splitter import.

6. Scalability Considerations

Large Documents

PyPDFLoader processes page-by-page — large PDFs are not loaded into memory at once. For 100+ document corpora, ChromaDB can be swapped for Qdrant (distributed) without changing any pipeline code — only the vectorstore initialization changes.

Increasing Query Load

The current architecture is single-threaded CLI. For production, the LangGraph graph is wrapped in a FastAPI async endpoint with a Celery/Redis request queue. Multiple workers share the same ChromaDB instance.

Latency Profile

Groq gives sub-second LLM inference. Query embedding: ~50ms (CPU). ChromaDB lookup: ~20ms. Total round-trip under 2 seconds for typical queries. Caching frequent query embeddings would reduce this further.

HITL at Scale

At scale, hitl_node sends the escalated query to a queue (Zendesk, Freshdesk, or custom dashboard). LangGraph's interrupt mechanism pauses the graph and resumes when the human submits a response via callback.